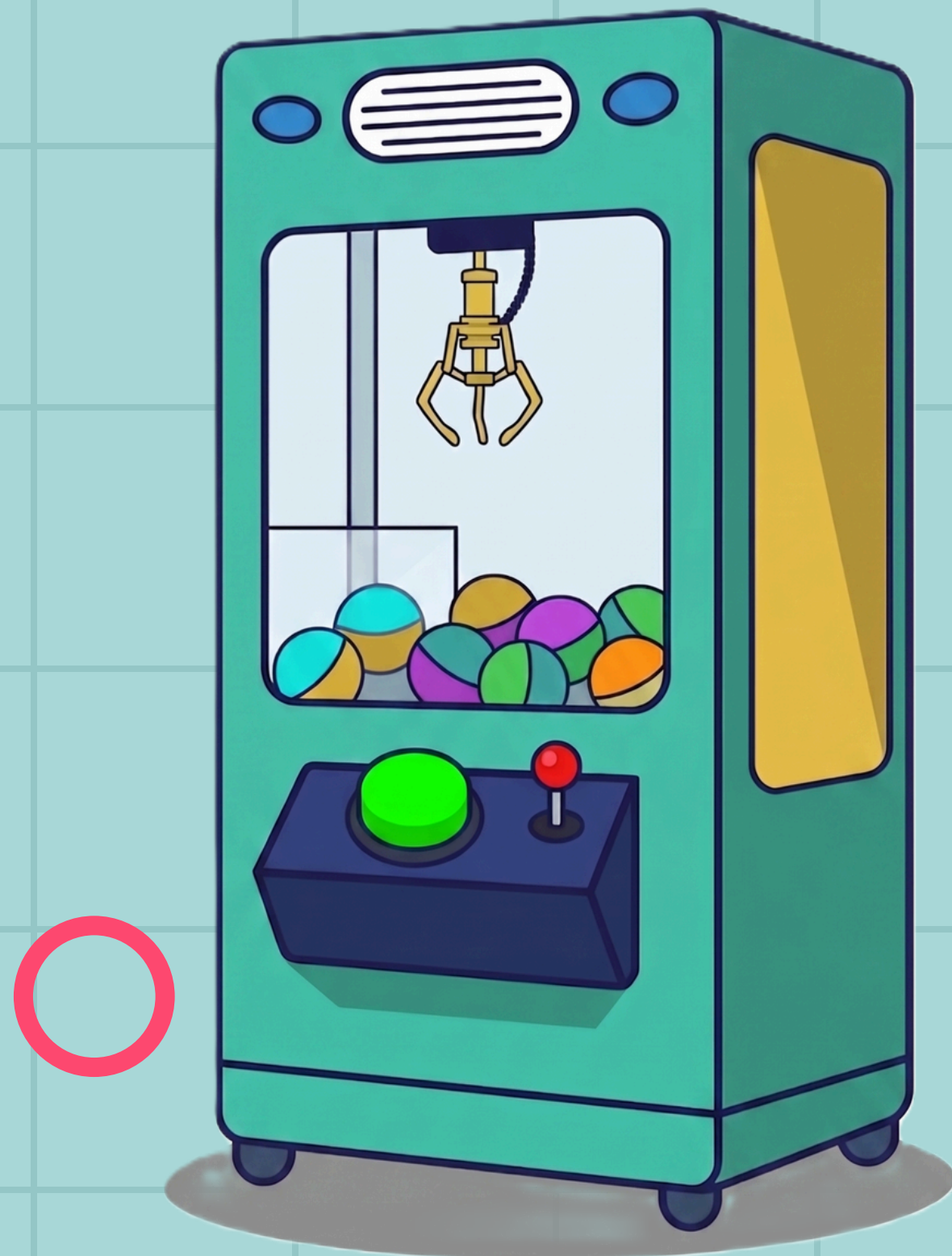
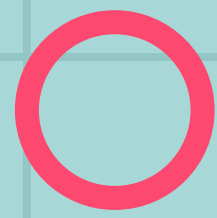




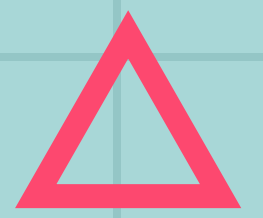
# CLAW MACHINE 3D

DANIEL NUNES  
NMEC: 120161

PROJETO FINAL-ICG-25/26



# LINK E QR CODE



[https://danuny21.github.io/ICG\\_Project/](https://danuny21.github.io/ICG_Project/)





# CONCEITO DO PROJETO

- Desenvolvimento de uma aplicação web 3D interativa que simula o clássico jogo de arcade "Claw Machine".
- As cápsulas contêm modelos low poly de criaturas animadas (animais, dinossauros e monstros).
- Controlar uma garra mecânica para extrair cápsulas de prémios.
- Explorar a coleção de prémios obtidos.





# FUNCAO

# ALIDADES

- **Mover a garra** (Setas / WASD / Joystick no mobile)
- **Tentar apanhar uma cápsula** (ESPAÇO/ IR no mobile)
- **Abrir cápsulas** e revelar os prémios (clicando)
- **Analisar a coleção** de 19 prémios (animais, dinossauros, monstros)
- **Ver o cenário** enquanto sentado numa mesa.
- **Alterar o tema** (lil-gui)
- **Alternar entre dia e noite** (lil-gui)





# BIBLIOTECAS E MÓDULOS



## Rapier3D (@dimforge/rapier3d-compatible)

- Motor de física que simula gravidade, peso e colisões realistas. Gere a interação entre a garra e as cápsulas



## lil-gui (lil-gui.module.min.js)

- Criação do painel lateral interativo para ajuste em tempo real da dificuldade e dos temas visuais e outras ações/opções.



## OrbitControls (three/addons/controls/OrbitControls.js)

- Controlo de câmaras permitindo a livre exploração 3D.



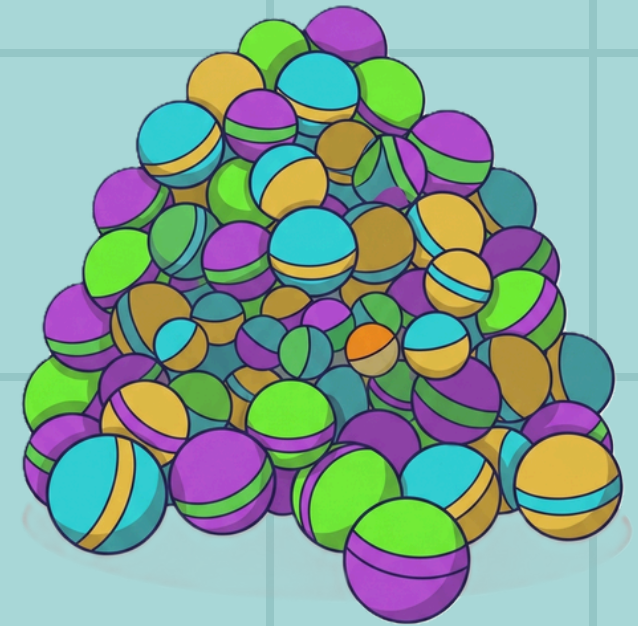
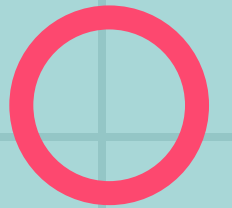
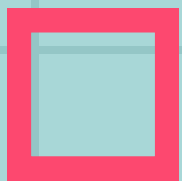
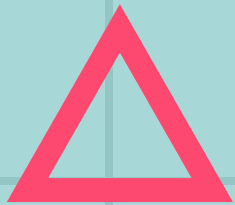
## SkeletonUtils (three/addons/utils/SkeletonUtils.js)

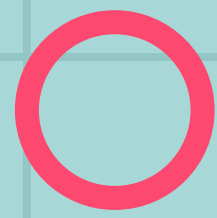
- Manipulação de malhas animadas (SkinnedMesh).



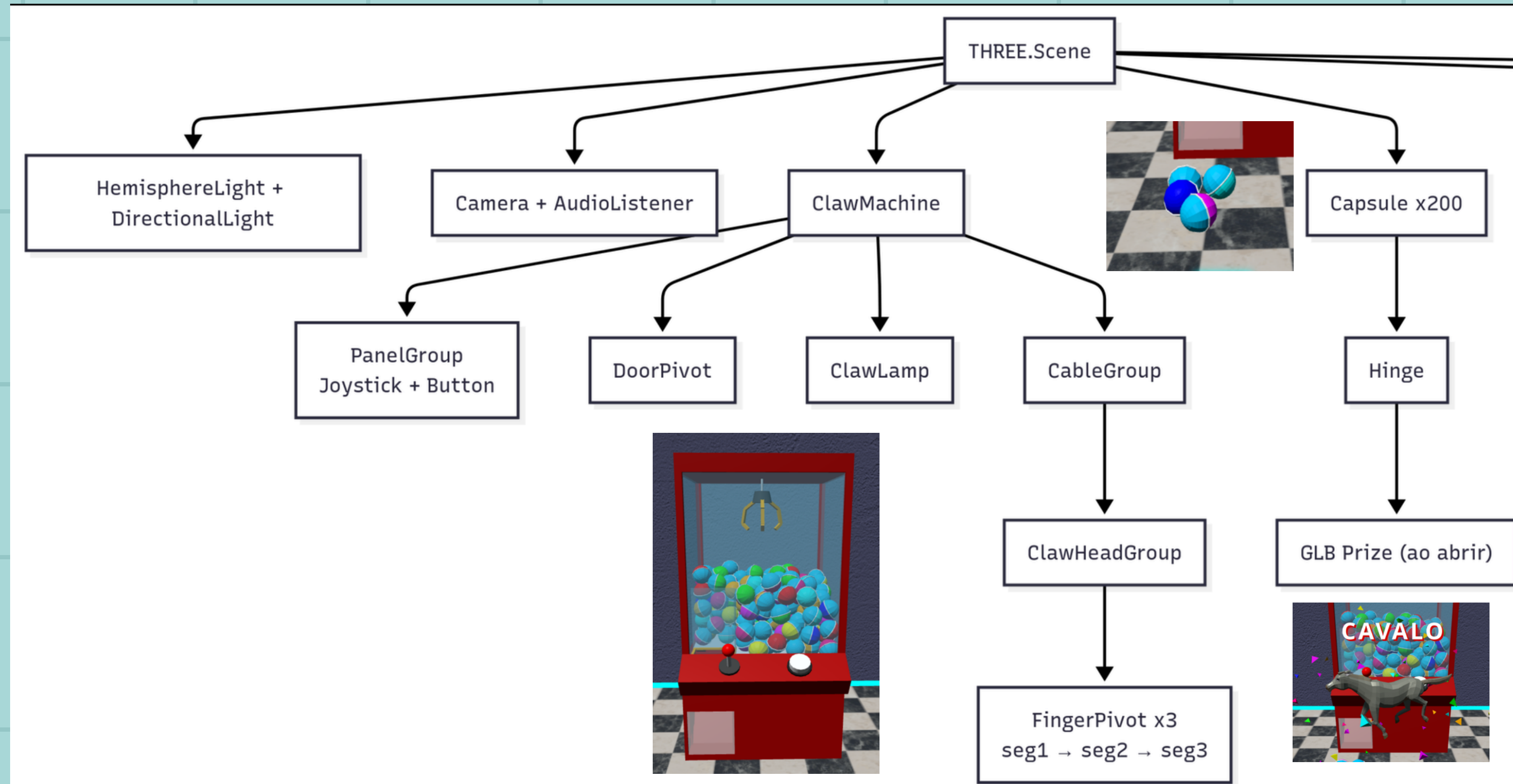
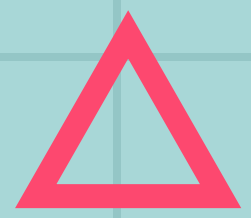
## GLTFLoader (three/addons/loaders/GLTFLoader.js)

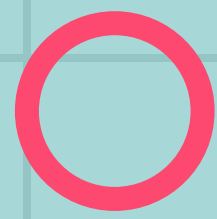
- Carregamento de modelos 3D no formato GLTF/GLB sendo o responsável por importar os prémios da coleção.



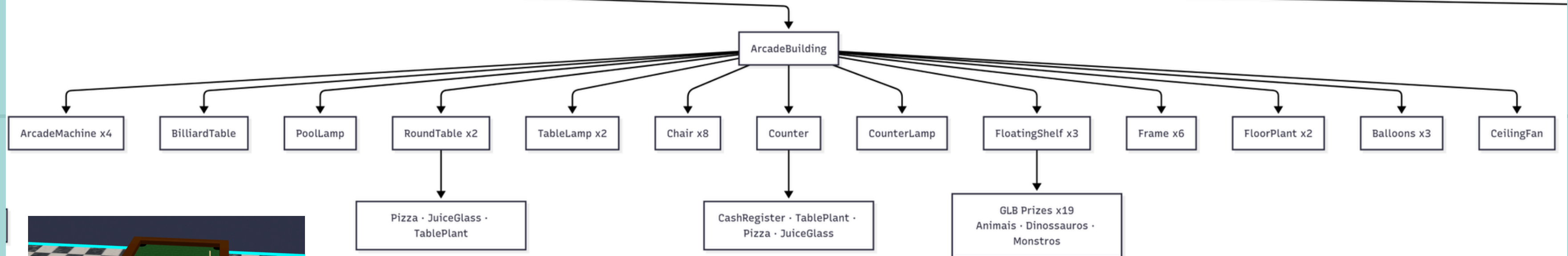
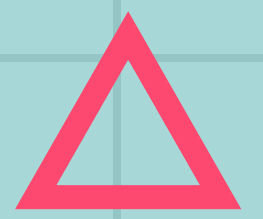


# GRAFO DA CENA 1

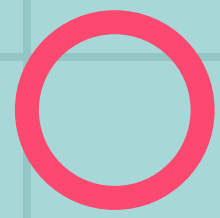




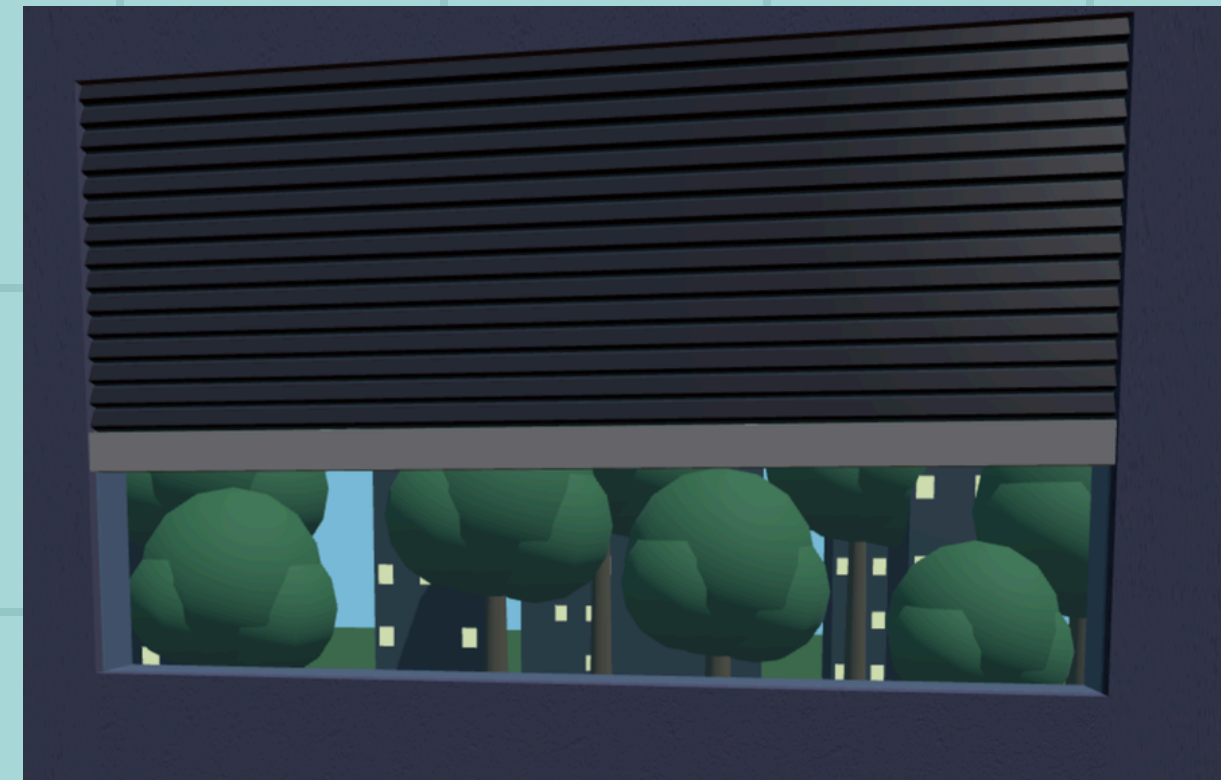
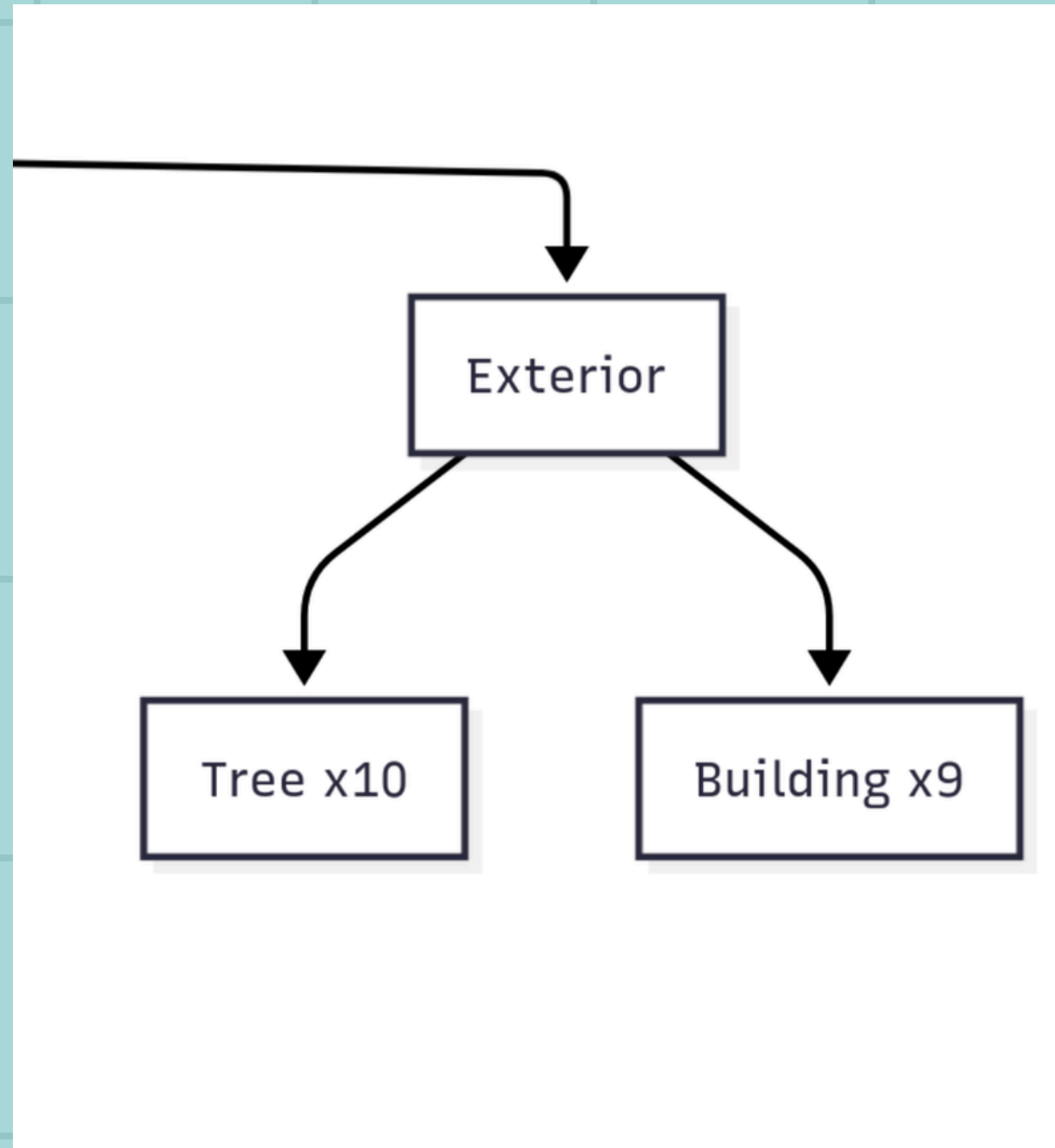
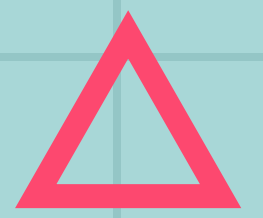
# GRAFO DA CENA 2





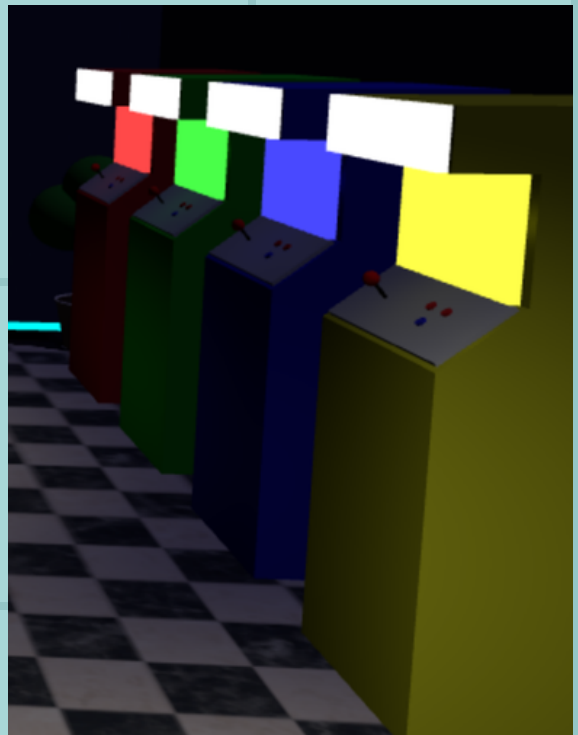
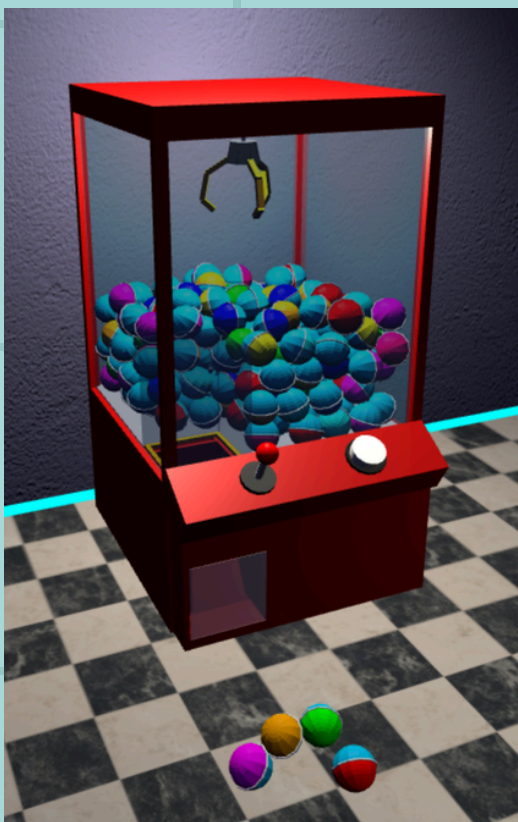
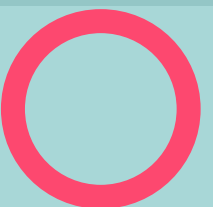


# GRAFO DA CENA 3



# MATERIAIS

| Material                     | Onde                                      | Porquê                                                            |
|------------------------------|-------------------------------------------|-------------------------------------------------------------------|
| MeshStandardMaterial         | Estrutura da máquina de garras            | Roughness e metalness para aspeto metálico realista               |
| MeshPhongMaterial            | Vidros, chão, paredes, cápsulas, edifício | Mais leve que Standard; suporta shininess, specularMap, normalMap |
| MeshPhongMaterial (emissivo) | Néons do edifício, ecrãs das arcades      | Emissive + emissiveIntensity para brilho sem depender de luz      |
| MeshBasicMaterial            | Silhuetas negras dos prémios bloqueados   | Sem cálculo de iluminação pois é cor pura                         |



○

ILUMINAÇÃO

×

| Luz                           | Tipo                                | Cor                                                                                         | Intensidade              |
|-------------------------------|-------------------------------------|---------------------------------------------------------------------------------------------|--------------------------|
| HemisphereLight               | Ambiente global                     | Céu Azul Claro com Chão Castanho (dia)<br>-----<br>Céu Azul Escuro com Chão Cizento (noite) | 0.6 (dia) / 0.35 (noite) |
| DirectionalLight              | Sol / Lua                           | Branco (dia)<br>Azul Arroxeadado (noite)                                                    | 3.0 (dia) / 0.5 (noite)  |
| SpotLight (candeeiros)        | Foco pendente                       | Branco                                                                                      | 0 (dia) / 500 (noite)    |
| PointLight (candeeiros)       | Preenchimento interior do candeeiro | Branco                                                                                      | 0 (dia) / 150 (noite)    |
| PointLight (interior máquina) | Iluminação interior da garra        | Branco                                                                                      | 80 (dia) / 2000 (noite)  |

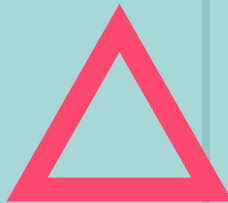
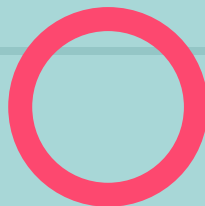


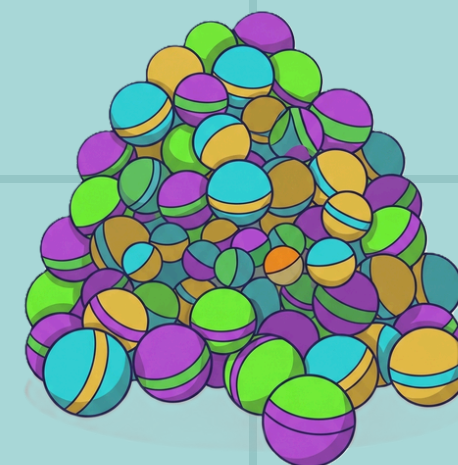
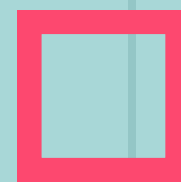
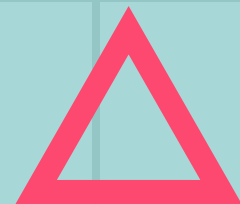
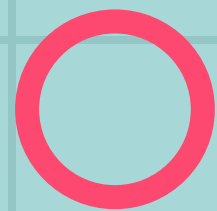


# TEXTURAS

| Textura           | Superfície          | Mapas usados                                                               |
|-------------------|---------------------|----------------------------------------------------------------------------|
| Tiles074          | Chão do arcade      | map (Color), normalMap (NormalGL), specularMap (Roughness)                 |
| PaintedPlaster017 | Paredes             | map, normalMap, specularMap                                                |
| Wood066           | Mesa de bilhar      | map, normalMap, specularMap (Roughness)                                    |
| Fabric021         | Cadeiras            | map, normalMap                                                             |
| Metal050A         | Elementos metálicos | map, normalMap, metalnessMap, roughnessMap                                 |
| Plastic017B       | Cápsulas            | normalMap, specularMap (Roughness)                                         |
| Posters (6x)      | Molduras na parede  | map - Donkey Kong, Pac-Man, Space Invaders, Sonic, Pinball, Street Fighter |

- Importadas do [ambientCG](#)





# ANIMAÇÕES

- **Sequência da Cápsula:** Implementada através de uma máquina de estados no loop manual. As transições usam interpolação linear para o movimento, combinado com a rotação e um efeito de dissolve para revelar o prêmio.
- **Animação dos Prêmios:** Recurso ao AnimationMixer nativo do Three.js para ler e reproduzir os clips de animação dos ficheiros GLB.
- **Sistema de Confetis:** Simulação física customizada calculada em tempo real com aplicação de gravidade contínua e rotações aleatórias por eixo.
- **Joystick e Botões:** Feedback baseado nos inputs do teclado. Utiliza interpolação linear (lerp) no loop de renderização para garantir que os movimentos dos comandos físicos da máquina são suaves e reativos.
- **Sistema da Garra:** Ciclo de vida gerido por uma máquina de estados completa. Combina cálculos de colisão, escalamento do cabo e efeitos de “shake”.







```
function animate(time) {  
  if (stats) stats.update();  
  requestAnimationFrame(animate);  
  
  const delta = (time - lastFrameTime) / 1000;  
  lastFrameTime = time;  
  
  if (prizeInspector?.state === "IDLE") cameraManager.update();  
  
  collectionManager.update(delta);  
  if (prizeInspector) prizeInspector.update(delta);  
  
  if (arcadeBuilding.fan) arcadeBuilding.fan.rotation.y += 2.0 * delta;  
  
  const clawResult = updateClawAnimation(gameState, animTime, clawMachine, keys, moveLimits, moveSpeed);  
  gameState = clawResult.newState;  
  animTime = clawResult.newTime;  
  
  physicsWorld.update(capsules, clawMachine);  
  capsuleOpener.update(time);  
  confetti.update();  
  
  controls.update();  
  renderer.render(scene, camera);  
}
```





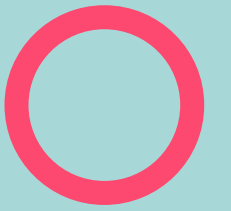


```
// Por na posição de descanso
function restClaw(clawMachine) {
  clawMachine.fingers.forEach(f => {
    f.rotation.x = THREE.MathUtils.lerp(f.rotation.x, -Math.PI / 3.2, 0.02);
  });
}

// Abrir a garra (para descer)
function openClaw(clawMachine) {
  clawMachine.fingers.forEach(f => {
    f.rotation.x = THREE.MathUtils.lerp(f.rotation.x, -Math.PI / 2.2, 0.02);
  });
}

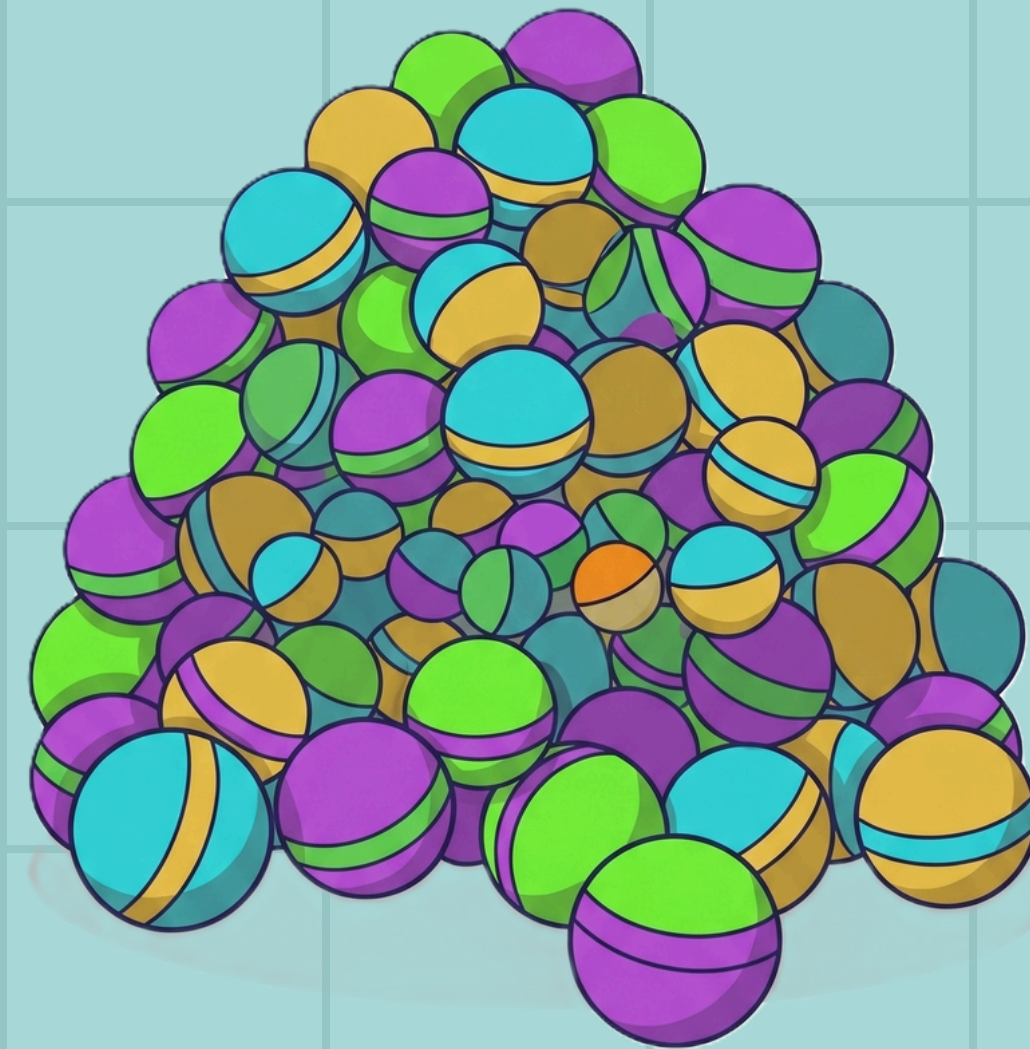
// Fechar a garra (depende da dificuldade)
function closeClaw(clawMachine) {
  const closeAngle = window.CONFIG_JOGO ? window.CONFIG_JOGO.gripRotation : -Math.PI / 7;
  clawMachine.fingers.forEach(f => {
    f.rotation.x = THREE.MathUtils.lerp(f.rotation.x, closeAngle, 0.02);
  });
}
```





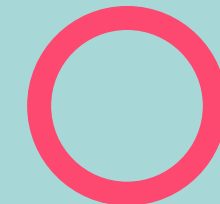
# CAMERA

- **PerspectiveCamera:** FOV 60° (desktop) / 85° (mobile) com near 0.1 e far 1000
- **Controlos:** OrbitControls com enableDamping = true, limites de distância, ângulo polar máximo e mínimo para não atravessar o chão e teto.
- **CameraManager com 3 vistas:**
  - Máquina: foca a máquina de garras, OrbitControls ativo
  - Coleção: foca as prateleiras, câmara fixa
  - Mesa: foca a zona das mesas, câmara fixa (mobile: orbita limitada)





# INTERAÇÕES



| Dispositivo | Tecla / Ação    | Função no Jogo                                  |
|-------------|-----------------|-------------------------------------------------|
| Teclado     | Setas / WASD    | Movimentar a Garra (Eixos X e Z)                |
| Teclado     | ESPAÇO          | Descer Garra / Abrir Cápsula / Sair da Inspeção |
| Rato        | Clique Esquerdo | Interagir com Cápsulas e Coleção                |
| Rato        | Drag / Scroll   | Controlo de Câmara (Orbit e Zoom)               |

| Controlo         | Mecânica    | Detalhes Técnicos                             |
|------------------|-------------|-----------------------------------------------|
| Joystick Virtual | Touch Drag  | Movimento 360° (Threshold 15px / Raio 50px)   |
| Botão de Ação    | Toque Único | Equivalente à tecla ESPAÇO                    |
| Deteção          | Automática  | Verificação de 'ontouchstart' no carregamento |





# DESENVOLVIMENTO

- **Ambiente:** VS Code + Live Server, sem bundlers.
- **Dependências:** Three.js e Rapier3D carregados dinamicamente pela rede.
- **Organização:**

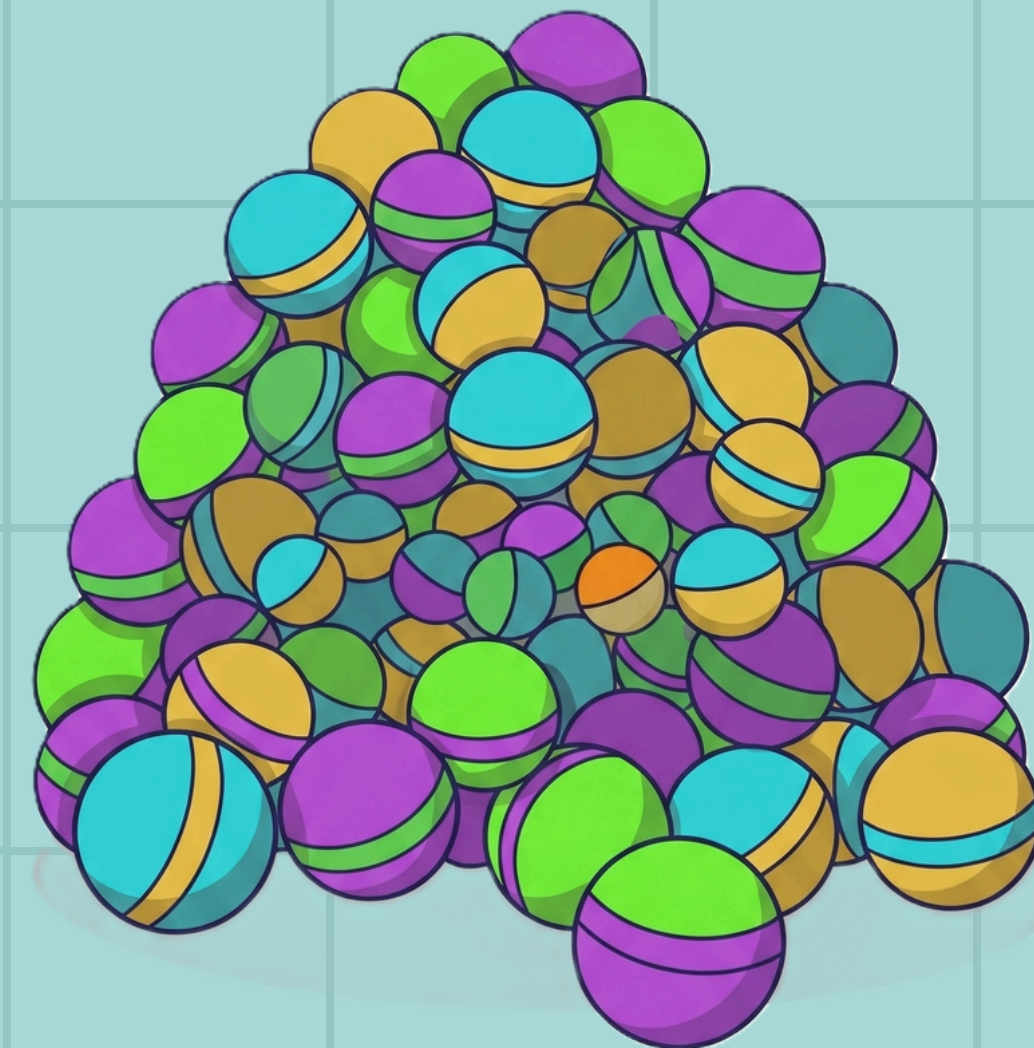
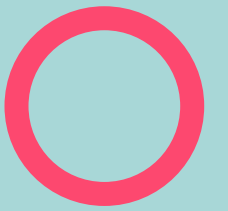
```
src/
├── js/
│   ├── main.js
│   ├── systems/
│   ├── models/
│   │   ├── glb/
│   │   ├── clawMachine.js
│   │   ├── arcadeBuilding.js
│   │   └── ...
│   ├── config/
│   └── textures/
├── sound/
├── styles/
│   └── style.css
└── index.html
```

# Orquestração: init, loop, game flow  
# Lógica operacional (física, interação, câmaras, animação)  
# Lógica de criação de Geometrias e Materiais do Three.js  
# Modelos 3D (Asset Pack - Kenney): Animais, dinossauros, monstros.  
# Máquina de garra  
# Edifício completo  
# capsule, mesa, cadeira, bilhar, etc.  
# Definições (cenário, iluminação, eixos, controlo de jogo)  
# Assets Mapas (Poly Haven): wood, fabric, metal, plastic, etc.  
# Assets de Áudio: efeitos sonoros (game start, recolhe prémios, musica background)  
  
# Interface, botões UI (menus), classes responsivas e tipografia  
# Carregamento do main.js



# FÍSICA

- O projeto usa **Rapier3D** em vez de qualquer física manual no Three.js.
- **O mundo físico corre em paralelo** com a cena 3D, a cada frame, o Rapier avança a simulação e os resultados são copiados de volta para os meshes Three.js.
- Os três tipos de corpos no projeto:
  - **fixed()**: geometria estática (paredes, chão, rampa, túnel)
  - **dynamic()**: as cápsulas que afetadas por gravidade e colisões
  - **kinematicPositionBased()**: dedos da garra e porta, movem-se mas controlados pelo código, não pela física

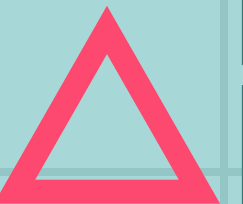




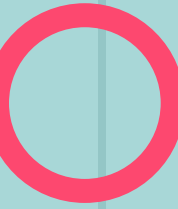
```
const W = this.world;
const staticBody = W.createRigidBody(RAPIER.RigidBodyDesc.fixed());
const quat = new THREE.Quaternion().setFromAxisAngle(new THREE.Vector3(0, 1, 0), this.baseRotY);

const colider = (desc, relPos) => {
  const pos = new THREE.Vector3(relPos.x, relPos.y, relPos.z).applyQuaternion(quat).add(this.basePos);
  return W.createCollider(
    desc.setTranslation(pos.x, pos.y, pos.z)
      .setRotation({ x: quat.x, y: quat.y, z: quat.z, w: quat.w })
      .setFriction(0.3)
      .setRestitution(0.3),
    staticBody
  );
};

// Base inferior da máquina
colider(RAPIER.ColliderDesc.cuboid(5.0, 0.475, 5.0), { x: -6.5, y: 0.5, z: 9.8 });
```

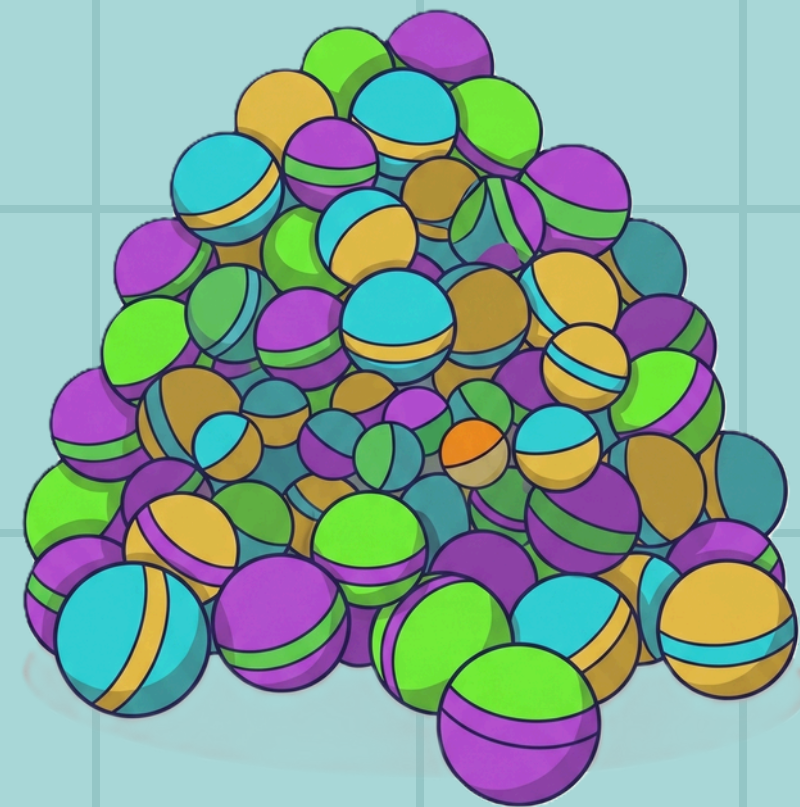
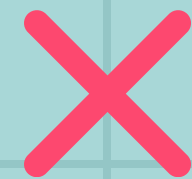


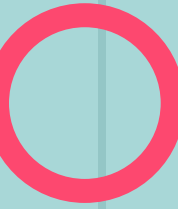




# DIFICULDADES

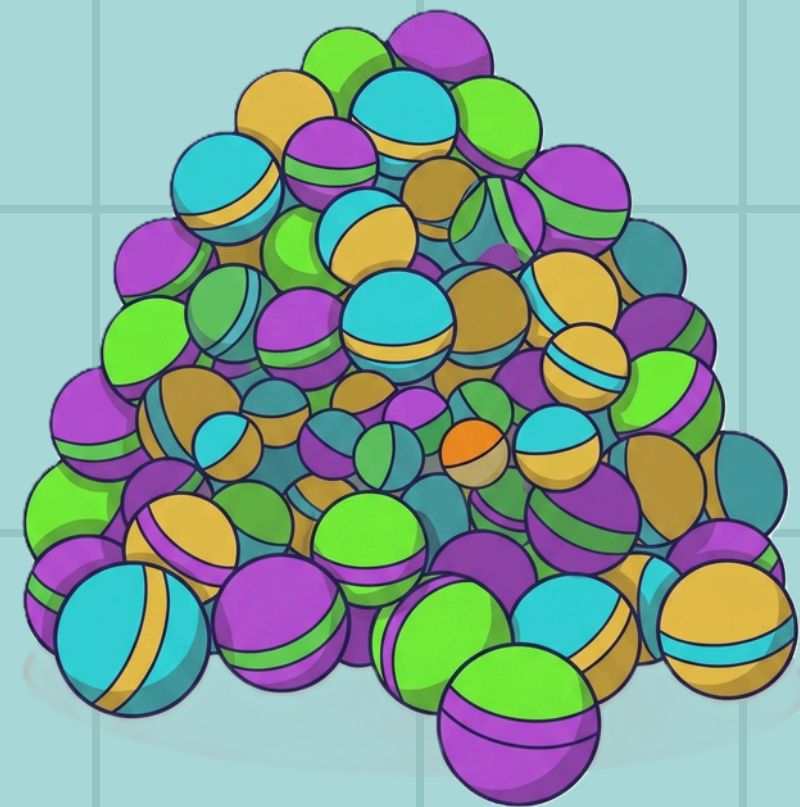
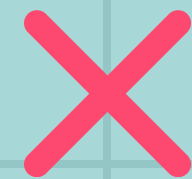
- **Sincronização Three.js com Rapier:** posições em espaço mundo vs. espaço local da máquina
- **Freeze ao abrir cápsulas:** carregar o modelo GLB na thread principal congela o ecrã durante uns segundos
- **Memory leak nas cápsulas abertas:** meshes e corpos Rapier acumulavam-se.
- **Limite de texture units WebGL (`MAX_TEXTURE_IMAGE_UNITS = 16`):** sombras de múltiplos SpotLights em simultâneo causavam problemas.





# USO DE IA

- **Sincronização Física/Visual:** Apoio no debugging e implementação da sincronização de coordenadas entre o motor Rapier3D e o Three.js.
- **Arquitetura Lógica:** Sugestão e estruturação das máquinas de estados responsáveis pelo ciclo de vida da garra e das cápsulas.
- **Gestão de Assets 3D:** Resolução de erros na importação de modelos no formato *.glb* e na execução das respectivas animações.





# PERFORMANCE

- **Pré-carregamento de GLBs (preloadAllPrizes):** Modelos são carregados em paralelo durante o loading screen em local storage e depois clonados.
- **Ciclo de vida completo das cápsulas:** Ao abrir a cápsula, remover também o Rigidbody do Rapier (causava Memory Leakage).
- **Pré-compilação de pedestal do PrizeInspector:** Pedestal usado na inspeção de prêmios é criado no construtor em vez de a cada clique.
- **Sombras opcionais:** toggle no lil-gui.
- **Vetores reutilizáveis no PhysicsSystem:** Vetores declarados no construtor para evitar alocações a cada frame.

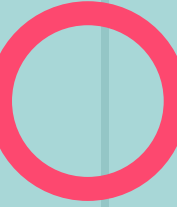
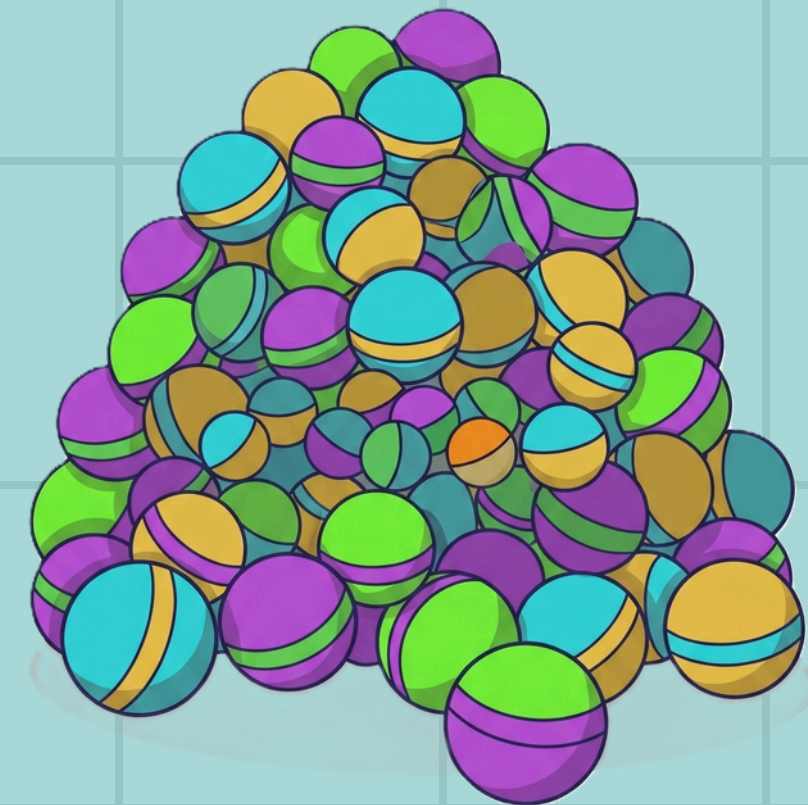
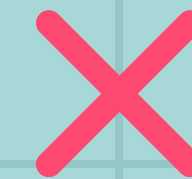






# CONCLUSÕES

- **Features mais importantes / interessantes:**
  - **Física real com Rapier3D:** cápsulas dinâmicas a colidir, dedos cinemáticos sincronizados frame a frame, porta com mola física.
  - **Coleção com 19 prémios animados:** sorteio ponderado por raridade, silhuetas para bloqueados, inspeção close-up, AnimationMixer por prémio
  - **Ciclo dia/noite + 2 temas em runtime:** todas as luzes e materiais mudam sem recriar nenhum objeto
- **O que faria diferente:**
  - **Separar o espaço de coordenadas da física do espaço visual desde o início:** o facto de a ClawMachine estar rodada obrigou a matemática extra em todo o lado.
  - **Biblioteca de máquinas de estado:** Em vez de implementar manualmente as transições, poderia ter usado uma biblioteca em que geria o diagrama de estados seria gerado automaticamente.



# REFERÊNCIAS



## Three.js Documentation:

Consulta da API principal para a implementação de geometrias, materiais, sistemas de iluminação, controlo de câmaras e loaders (carregadores de assets).



## Rapier3D Documentation:

- Documentação do motor de física baseado em WASM, essencial para a configuração do ambiente (World) e gestão de colisões e gravidade (RigidBody e Collider).



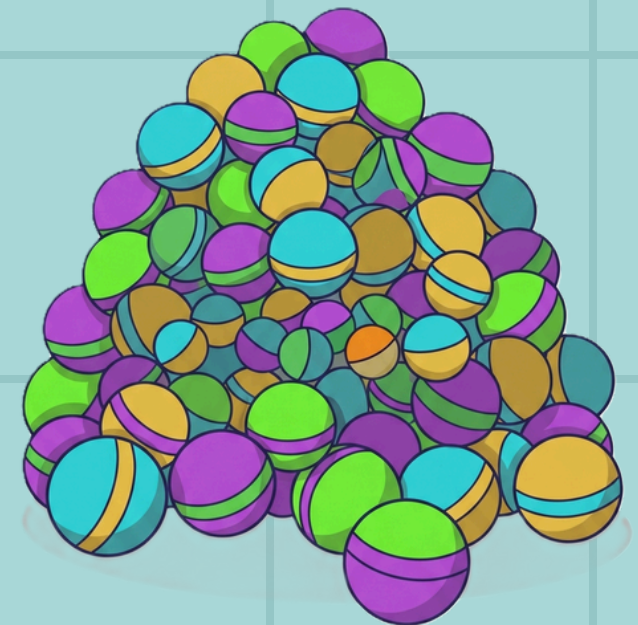
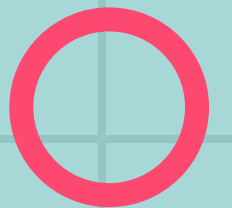
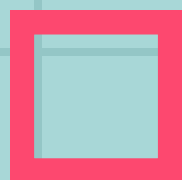
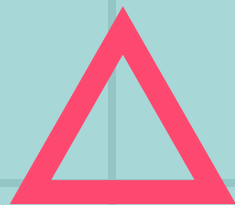
## poly.pizza - Quaternius

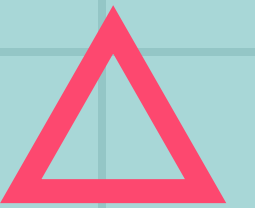
- 19 modelos (formato .glb) representativos de animais, dinossauros e monstros (sob licença CC0 - Domínio Público).



## ambientCG:

- Texturas aplicadas no cenário: Tiles074 (chão), PaintedPlaster017 (paredes), Wood066 (móvia), Fabric021 (cadeiras), Metal050A (elementos metálicos) e Plastic017B (cápsulas).

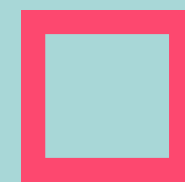
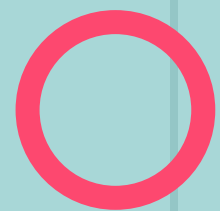




# LIVE DEMO







OBRIGADO

PELA



ATENÇÃO

